

Hybrid Computing Technology

1

Fundamental of Computer Processing and Computation

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

FACULTY OF INFORMATION TECHNOLOGY
KMITL

Topics

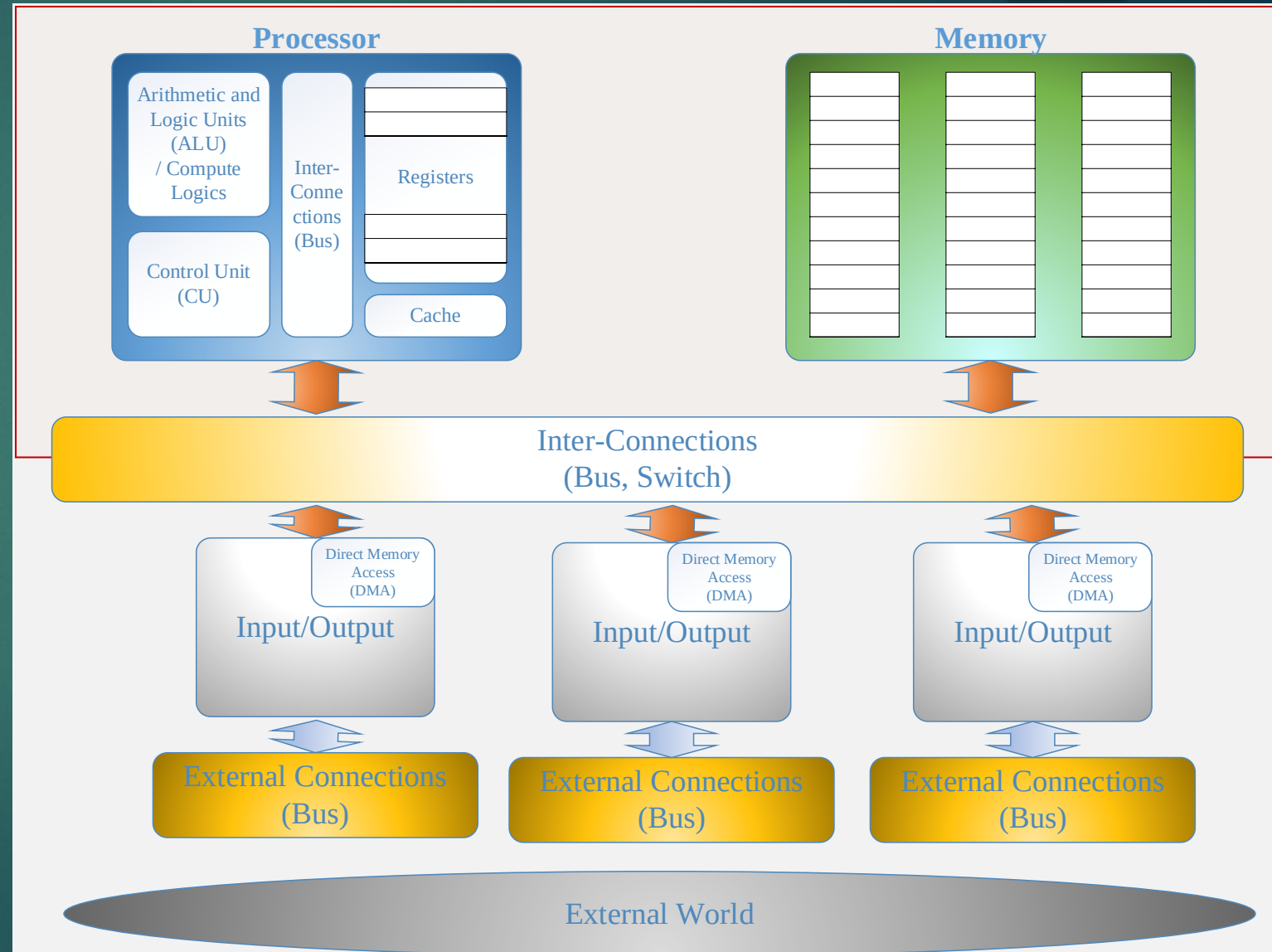
2

- ▶ Computer Architecture
- ▶ Computer Execution
- ▶ Computer Code Development
- ▶ Computing Technology
- ▶ Conclusion

Computer Architecture

3

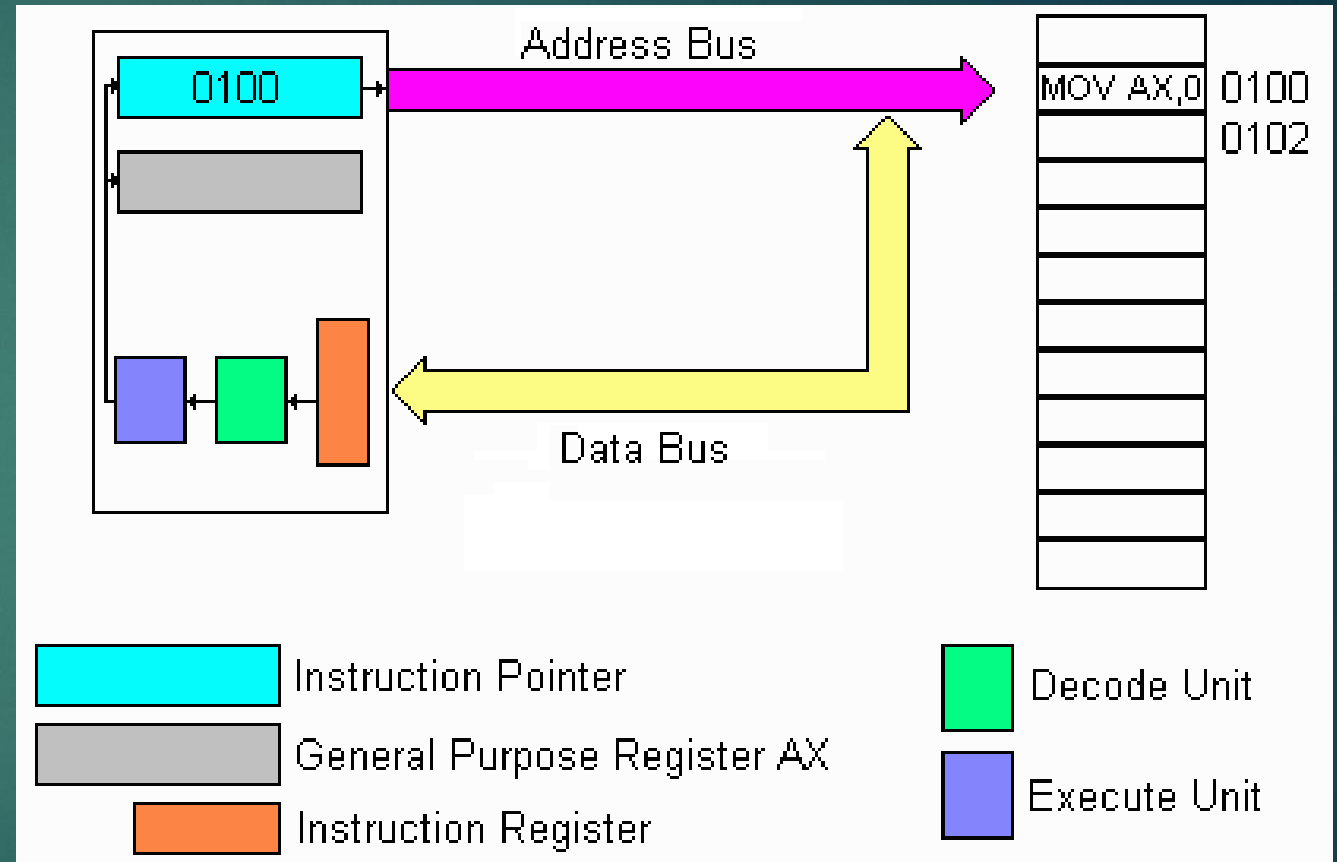
- ▶ Processor/CPU (Central Processing Unit)
 - ▶ Registers
 - ▶ Arithmetic and Logic Units (ALU)
 - ▶ Control Unit (CU)
 - ▶ Interconnections (Bus)
- ▶ Memory
- ▶ Input/Output
 - ▶ Direct Memory Access (DMA)
 - ▶ Interrupt
- ▶ Interconnections (BUS)



Computer Running

4

- ▶ Program => Set of Instructions
- ▶ Instruction Cycles
 - ▶ Fetch
 - ▶ Decode
 - ▶ Execute
- ▶ Fetching
 - ▶ [Instruction Pointer] => Address Bus
 - ▶ [Memory] => [Instruction Register]
 - ▶ Increment Instruction Pointer
- ▶ Decoding
 - ▶ Decode [Instruction Register]
- ▶ Executing
 - ▶ Execute Instruction by Sending Control Signals to Components

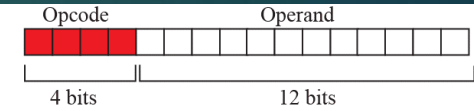


Instruction Types

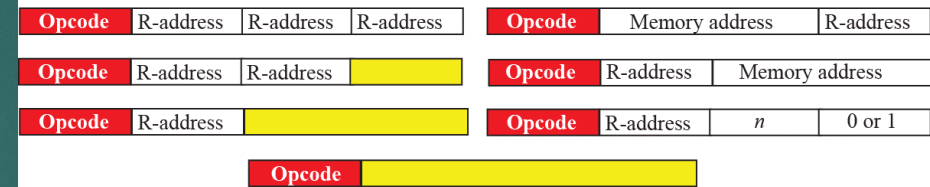
▶ Instruction Set Architecture (ISA)

Instruction Types

- ▶ Data handling and memory operations
 - ▶ Including input/output or hardware devices
- ▶ Arithmetic and logic operations
 - ▶ Number operations
- ▶ Control flow operations
- ▶ Coprocessor instructions

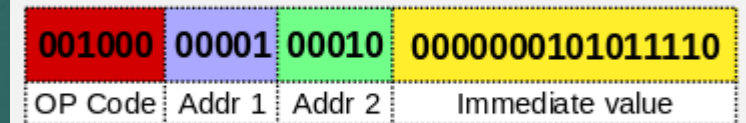


a. Instruction format



b. Instruction types

MIPS32 Add Immediate Instruction



Equivalent mnemonic: **addi \$r1, \$r2, 350**

Computer Working

► Types of works

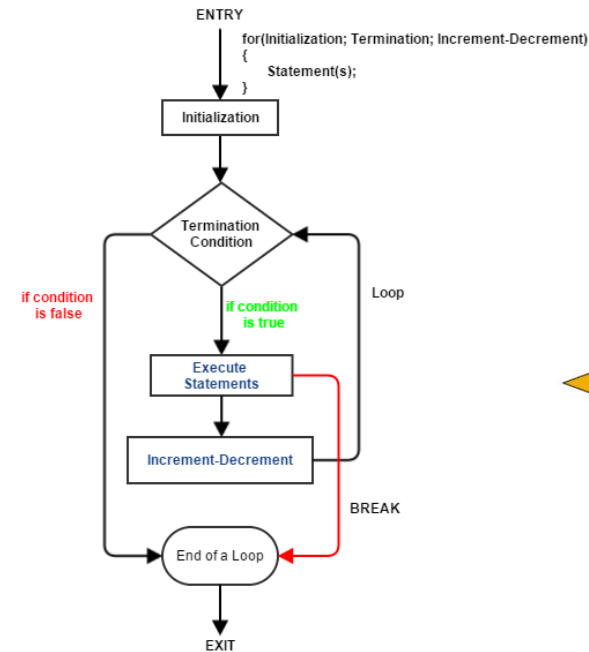
► Program Flow Control

► Number Processing

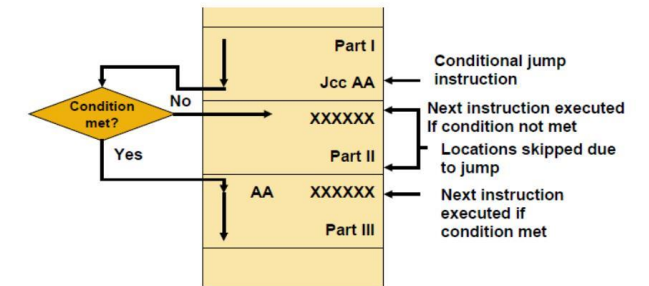
► => Computing

► More Digital Circuits

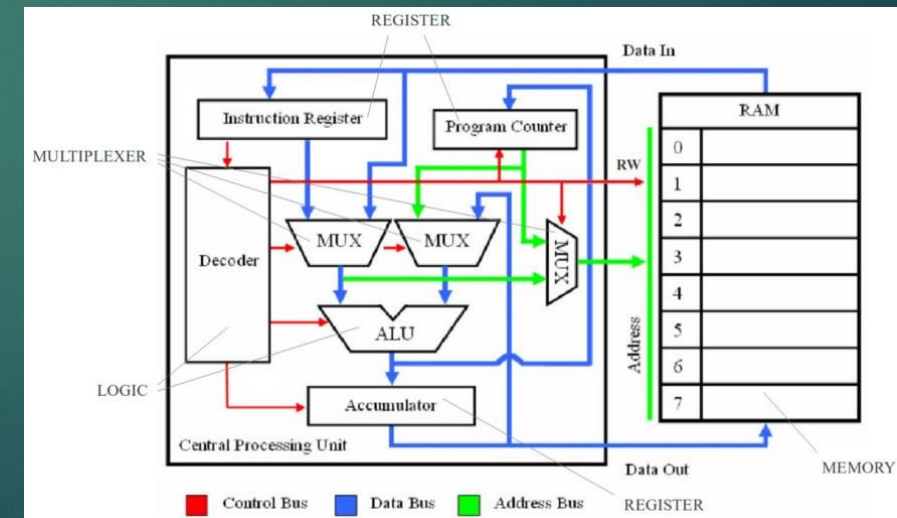
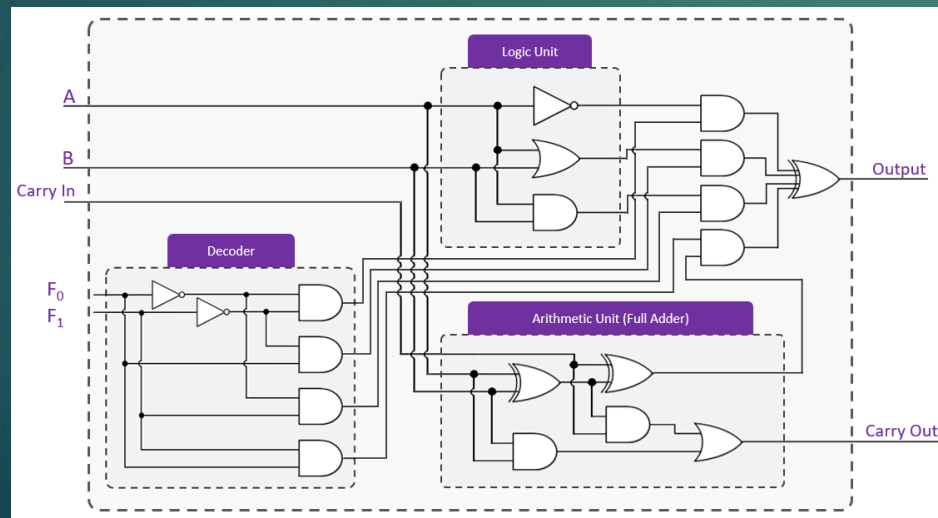
6



Conditional jump



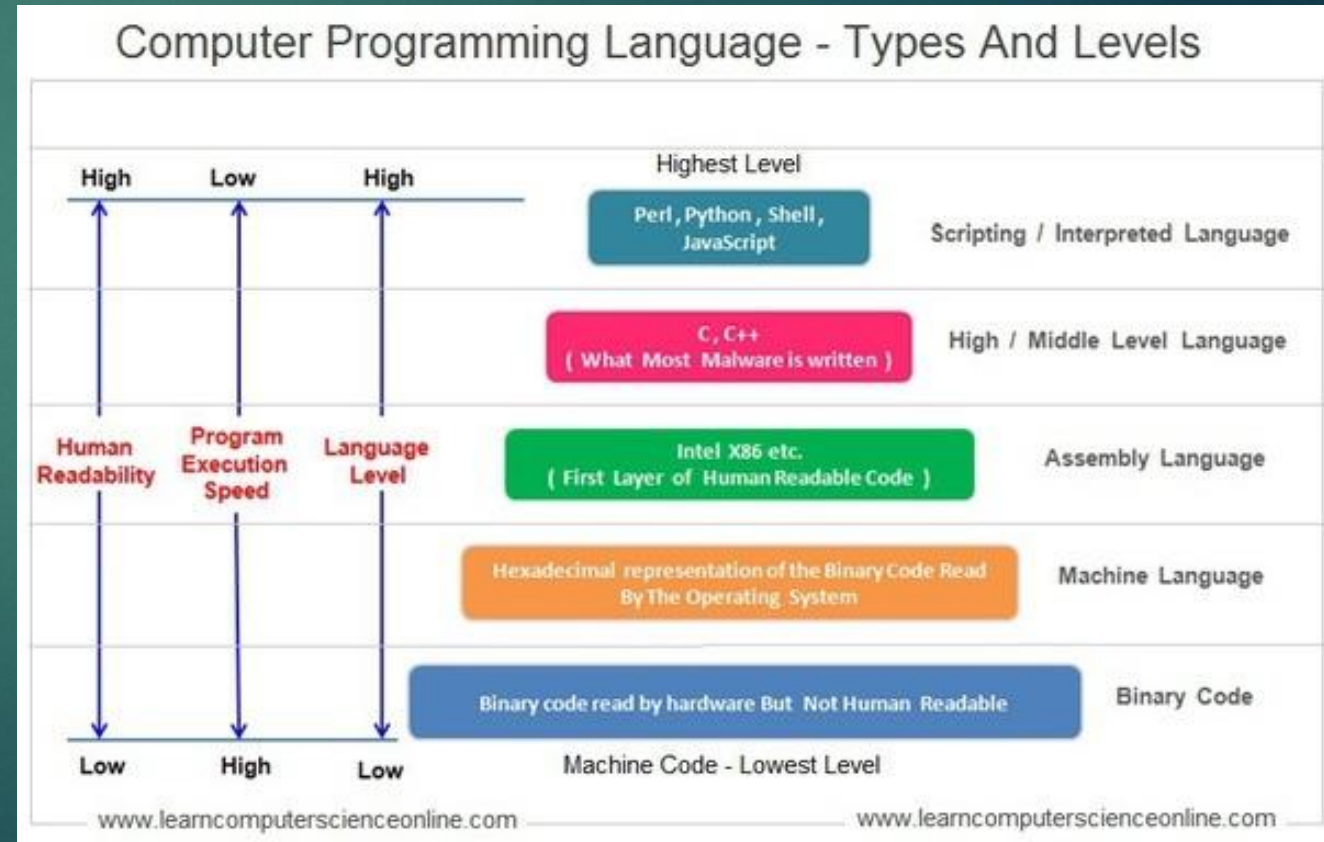
Conditional jump program sequence



Types of Codes

7

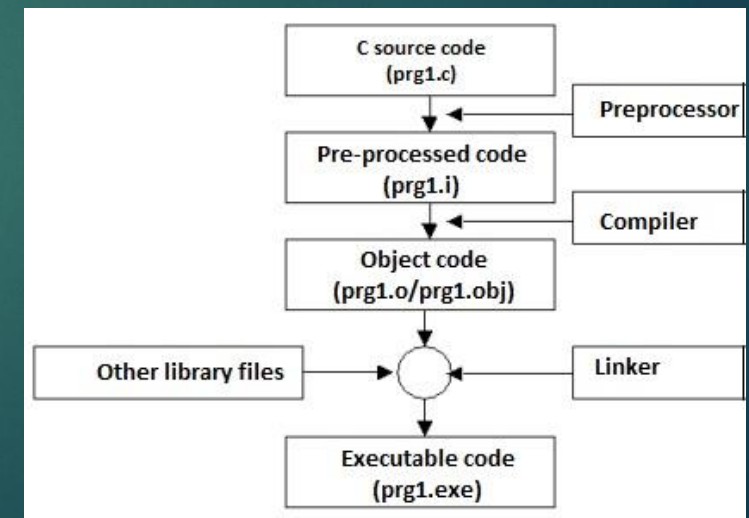
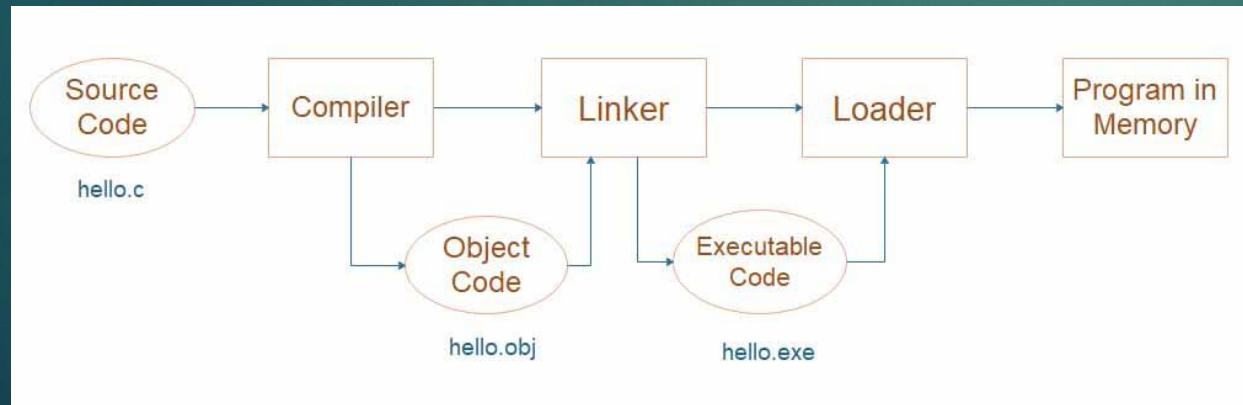
- ▶ Machine/Binary Codes => Instruction Codes for Execution
- ▶ Assembly Codes / Language => Represent Machine Codes for Human Reading
 - ▶ Assembler => Translator Assembly Codes to Machine Codes
- ▶ Low Level Programming Language
 - ▶ e.g. Assembly, C/C++
- ▶ Middle Level Programming Language
 - ▶ e.g. C/C++
- ▶ High Level Programming Language
 - ▶ e.g. Basic, Pascal, Python
 - ▶ Script



Codes Development

8

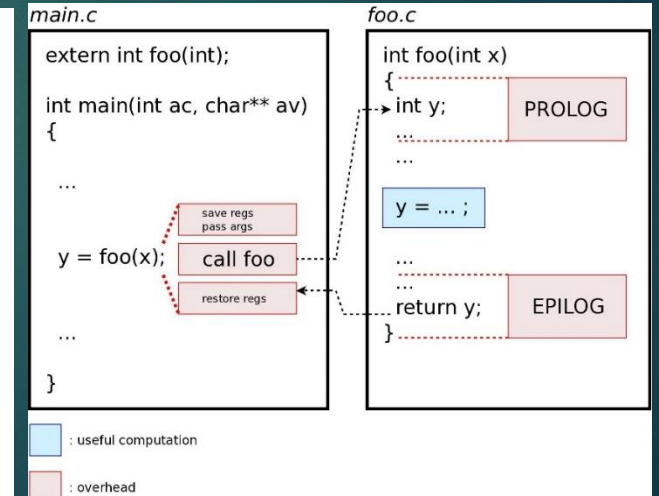
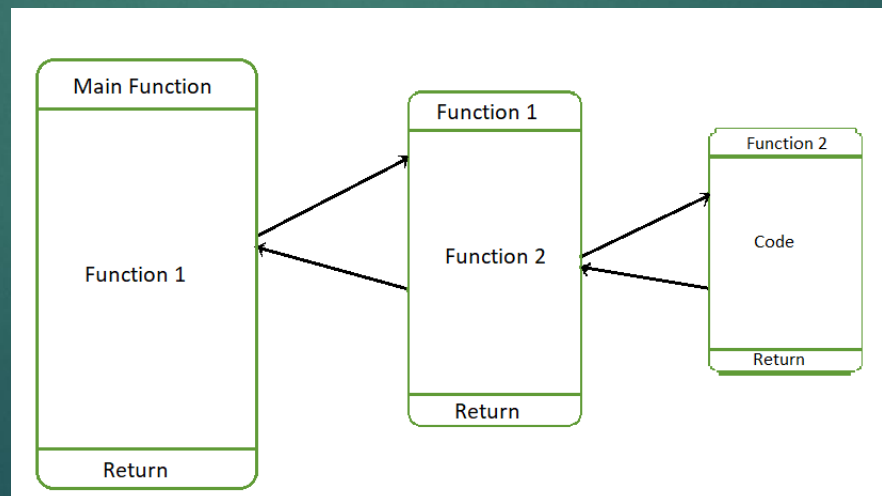
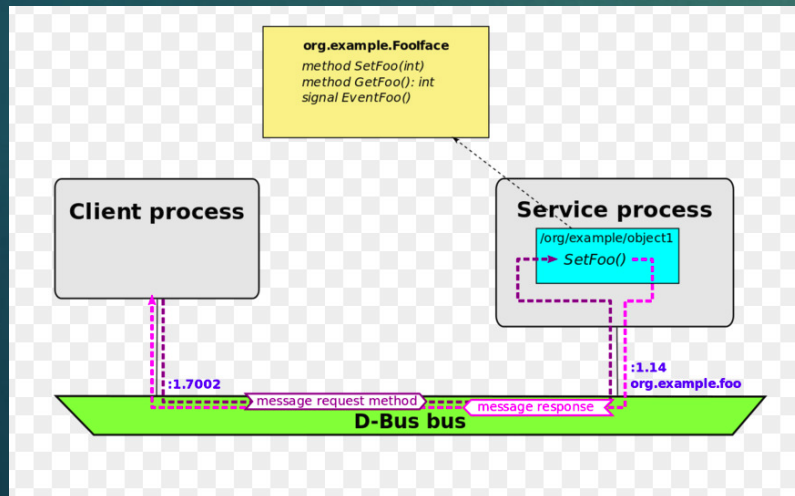
- ▶ Source Code => Programming Language
- ▶ **Compiler** => Translate Source Code to Object Code
- ▶ Object Code => Executable/Machine Code with Symbolic Links
- ▶ **Linker** => Link all Codes Together
- ▶ Executable Code => Code Ready for Executing, e.g. Machine Code



Code Execution

9

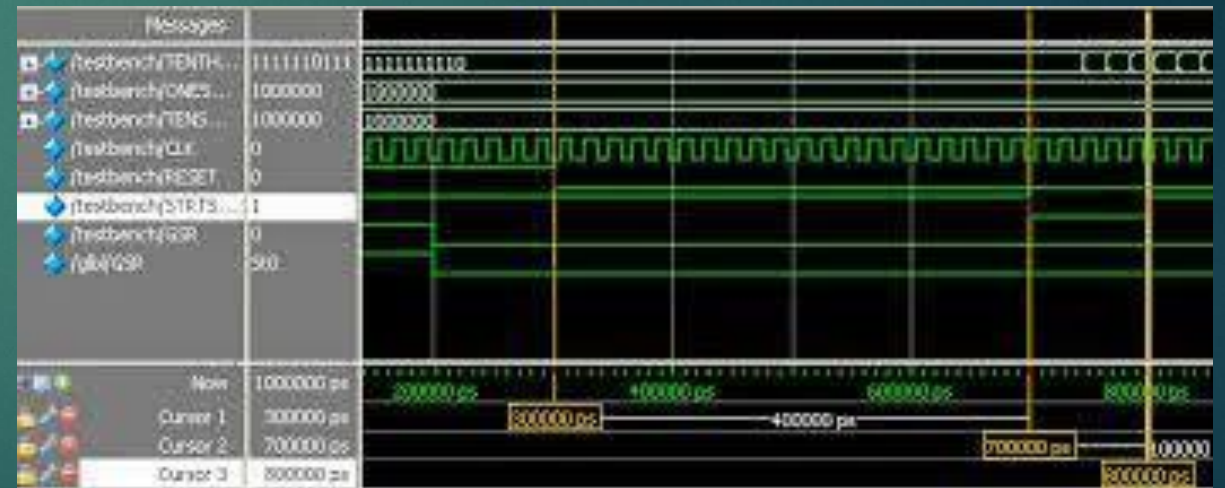
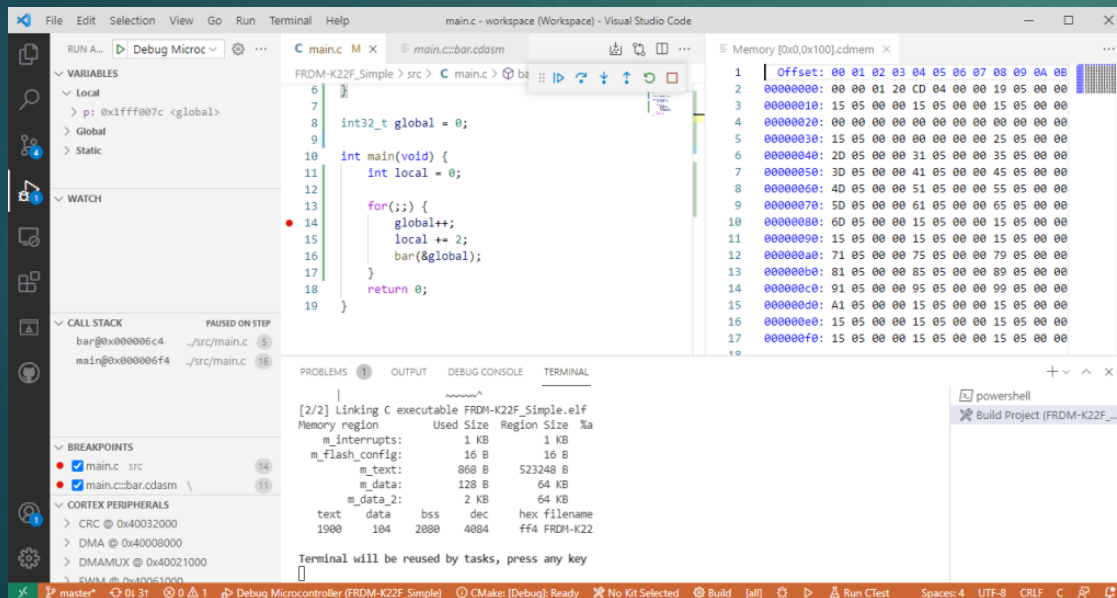
- ▶ Executable Code => Code Ready for Executing, e.g. Machine Code
- ▶ Code Loader => Loading Code into Executing Environment, e.g. Memory
- ▶ Code Trigger/Start => Activate Code to be executed, e.g. Procedure/Function Calling
- ▶ Code Executing => Codes are executing, e.g. Program running
- ▶ Code Ending => End of executing, may return somethings
- ▶ Code Unloader => Optional Remove Code from Executing Environment



Code Debugging

10

- ▶ Test and Verify Code execution
- ▶ See and Trace States of Execution, e.g. Variables
- ▶ Stepping for Code Execution => Verify execution steps

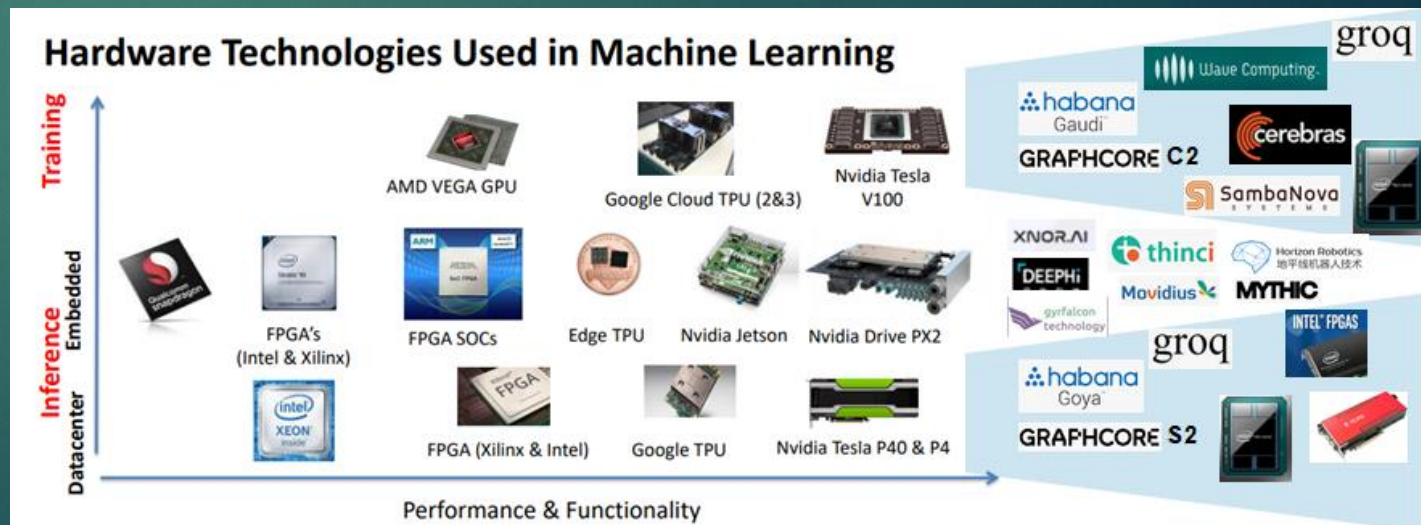
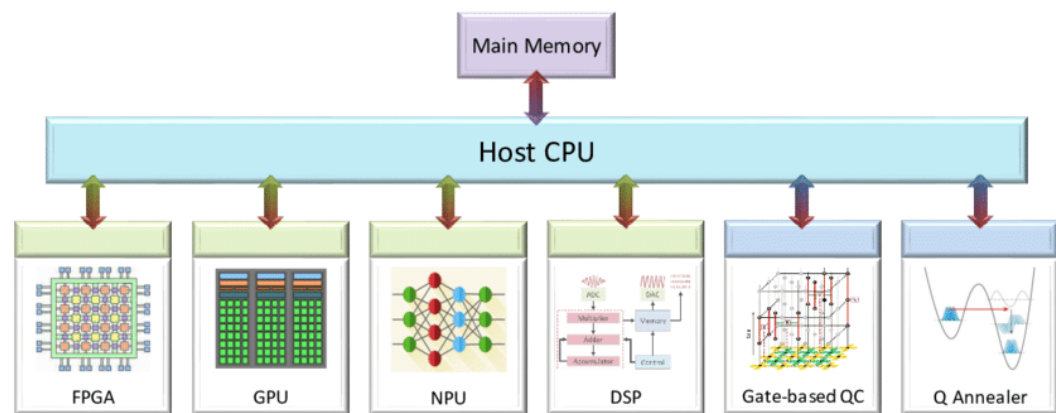


Computing Technology

11

- ▶ Number Processing in Faster Ways => Parallel Processing
- ▶ Accelerators in Main Host Computer => More Hardware/Circuits
- ▶ Host Computer => Control all activities
- ▶ Accelerators => Assist Host for high speed computing/processing

Classical host processor keeps the control over the total system and delegates the execution of the available accelerators.



Conclusion

- ▶ Computer for **Control and Computing**
- ▶ Computing Technology make computer **compute numbers faster**
- ▶ Faster computing by **Parallel Processing**
- ▶ Parallel Processing needs **More Hardware, Digital Circuits**
- ▶ **Codes** make for **more Parallel Executions**

END

QUESTIONS & ANSWERS